COMSATS UNIVERSITY ISLAMABAD



CSC211-Data Structures

Lab 2, 3 and 4:

Singly, Doubly and Circular Linked Lists

Name	Muhammad Awais
Registration	FA25-BSE-057
Class	BSE-3B
Instructor's Name	Mr. Zahid Anwar
Date	October 4, 2026

LAB 2

SINGLY LINKED LIST

ACTIVITY 1

```
#include <iostream>
using namespace std;

struct Nodetype {
   int data;
   Nodetype* next;
};

Nodetype* first = NULL;
Nodetype* last = NULL;

void insert_end() {
   Nodetype* p;

   p = new Nodetype;

   cout << "Enter the data in node: ";
   cin >> p->data;

   p->next = NULL;

   if (first == NULL) {
      first = last = p;
   }
   else {
      last->next = p;
```

```
        last = p;
    }
}

void display() {
    Nodetype* temp = first;

    cout << "Linked List: ";

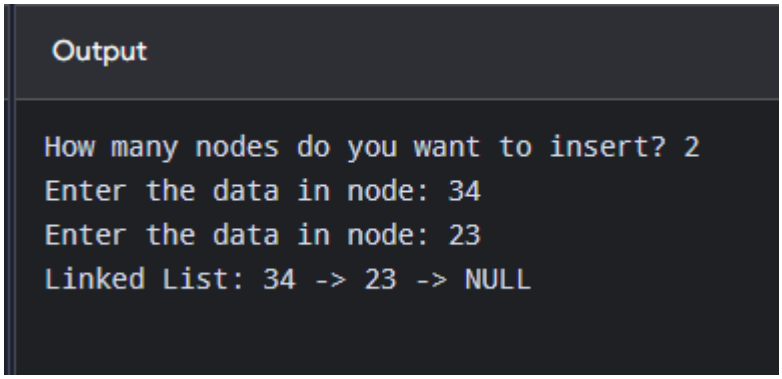
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

int main() {
    int n;

    cout << "How many nodes do you want to insert? ";
    cin >> n;

    for (int i = 0; i < n; i++) {
        insert_end();
    }
    display();
    return 0;
}
```



ACTIVITY 2

```
#include <iostream>

using namespace std;


struct Nodetype {
   int data;
   Nodetype* next;
};


Nodetype* first = NULL;
Nodetype* last = NULL;


void insert_start(){
   Nodetype* p;
   p = new Nodetype;



   cout << "Enter the data in node: ";
   cin >> p->data;


   if (first == NULL)
   {
      first = last = p;
```

```
        p->next = NULL;
    }
    else
    {
        p->next = first;
        first = p;
    }
}

void display(){
    Nodetype* temp = first;

    cout << "Linked List: ";

    while (temp != NULL)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

int main(){
    int n;

    cout << "How many nodes do you want to insert? ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
```

```
        insert_start();
    }

    display();
    return 0;
}
```

Output

```
How many nodes do you want to insert? 3
Enter the data in node: 14
Enter the data in node: 15
Enter the data in node: 13
Linked List: 13 -> 15 -> 14 -> NULL
```

ACTIVITY 3

```
#include <iostream>
using namespace std;

struct Nodetype{
    int data;
    Nodetype* next;
};

Nodetype* first = NULL;
Nodetype* last = NULL;

// Insert node at the end
void insert_end(){
    Nodetype* p = new Nodetype;
```

```
    cout << "Enter data: ";
    cin >> p->data;

    p->next = NULL;

    if (first == NULL){
        first = last = p;
    }
    else{
        last->next = p;
        last = p;
    }
}

// Search for a value
Nodetype* search(int key){
    Nodetype* p = NULL;

    p = first;
    while (p != NULL && p->data != key)
    {
        p = p->next;
    }
    return p;
}

int main()
{
    int n, key;

    cout << "How many nodes? ";
```

```
    cin >> n;

    for (int i = 0; i < n; i++)
    {
        insert_end();
    }

    cout << "Enter value to search: ";
    cin >> key;

    Nodetype* result = search(key);

    if (result == NULL)
    {
        cout << "Value not found." << endl;
    }
    else
    {
        cout << "Value found: " << result->data << endl;
    }

    return 0;
}
```

Output

```
How many nodes? 3
Enter data: 6
Enter data: 5
Enter data: 8
Enter value to search: 8
Value found: 8
```

ACTIVITY 4

```
#include <iostream>
using namespace std;

struct Nodetype{
   int data;
   Nodetype* next;
};

Nodetype* first = NULL;
Nodetype* last = NULL;

void insert_end(){
   Nodetype* p;

   p = new Nodetype;

   cout << "Enter data: ";
   cin >> p->data;

   p->next = NULL;

   if (first == NULL){
      first = last = p;
   }
   else{
      last->next = p;
      last = p;
   }
}
```

```
Nodetype* search(int key){
   Nodetype* p;
   p = first;

   while (p != NULL && p->data != key){
      p = p->next;
   }

   return p;
}

void insert_after(int key)
{
   Nodetype* p = NULL;
   p = search(key);

   if (p == NULL){
      cout << "Value not found." << endl;
   }
   else{

      Nodetype* Newnode;
      Newnode = new Nodetype;

      cout << "Enter data for new node: ";
      cin >> Newnode->data;

      Newnode->next = p->next;
      p->next = Newnode;

      if (p == last){
```

```
            last = Newnode;
        }

        cout << "New node linked successfully." << endl;
    }
}

void display(){
    Nodetype* p = first;

    cout << "\nLinked List: ";

    while (p != NULL){
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

int main(){
    int n;
    int key;

    cout << "How many nodes do you want to insert? ";
    cin >> n;

    for (int i = 0; i < n; i++){
        insert_end();
    }
```

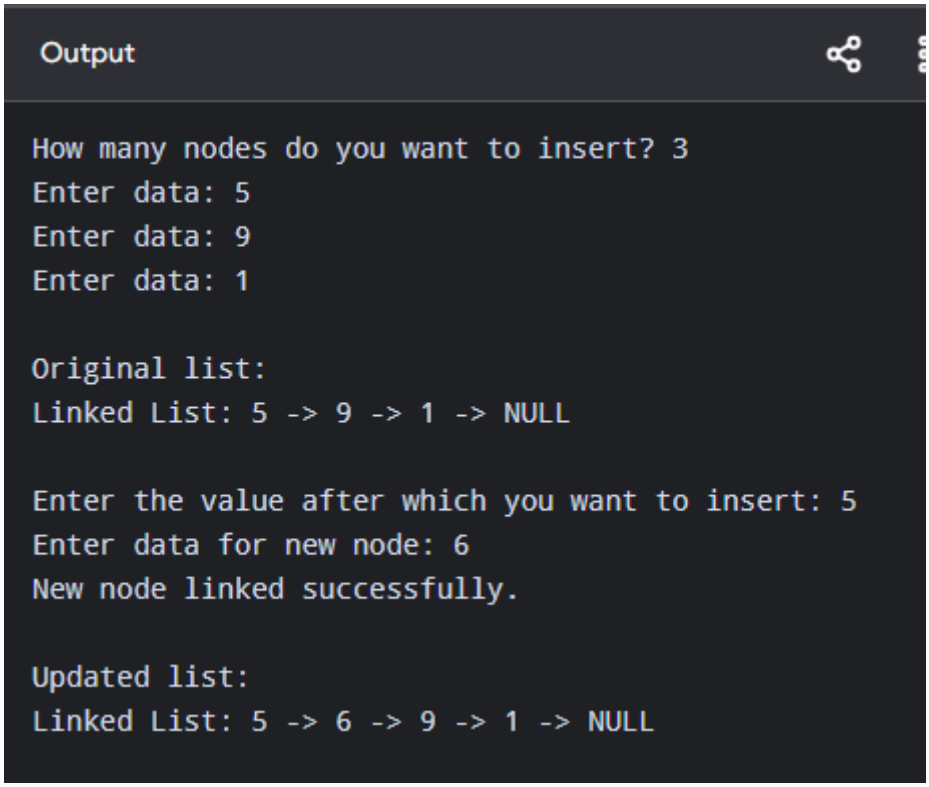
```
    cout << "\nOriginal list:";
    display();

    cout << "\nEnter the value after which you want to insert: ";
    cin >> key;

    insert_after(key);

    cout << "\nUpdated list:";
    display();

    return 0;
}
```



ACTIVITY 5

```
#include <iostream>
using namespace std;

struct Nodetype{
   int data;
   Nodetype* next;
};

Nodetype* first = NULL;
Nodetype* last = NULL;

void insert_end()
{
   Nodetype* p = new Nodetype;

   cout << "Enter data: ";
   cin >> p->data;

   p->next = NULL;

   if (first == NULL)
   {
      first = last = p;
   }
   else
   {
      last->next = p;
      last = p;
   }
}
```

```
void delete_first()
{
   Nodetype* p;

   if (first == NULL)
   {
      cout << "\nLinked List is empty." << endl;
   }
   else
   {

      p = first;

      first = first->next;

      if (first == NULL)
      {
         last = NULL;
      }

      delete p;

      cout << "\nFirst node deleted successfully." << endl;
   }
}

void display()
{
   Nodetype* p = first;
```

```
    cout << "\nLinked List: ";

    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

int main()
{
    int n;

    cout << "How many nodes do you want to insert? ";
    cin >> n;

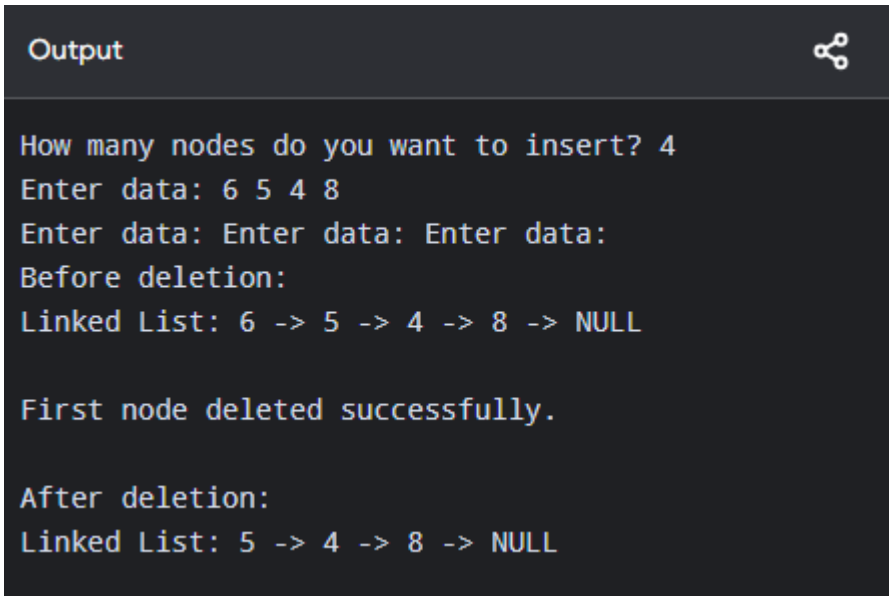
    for (int i = 0; i < n; i++)
    {
        insert_end();
    }

    cout << "\nBefore deletion:";
    display();

    delete_first();

    cout << "\nAfter deletion:";
    display();
```

```
    return 0;
}
```



ACTIVITY 6

```
#include <iostream>
using namespace std;

struct Nodetype
{
    int data;
    Nodetype* next;
};

Nodetype* first = NULL;
Nodetype* last = NULL;


void insert_end()
{
```

```
    Nodetype* p = new Nodetype;

    cout << "Enter data: ";
    cin >> p->data;

    p->next = NULL;

    if (first == NULL)
    {
        first = last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
}

void delete_last()
{
    Nodetype* q;
    Nodetype* q1;

    q1 = NULL;
    q = first;

    if (q == NULL)
    {
        cout << "\nLinked List is empty." << endl;
    }
    else
```

```
    {
        while (q != last)
        {
            q1 = q;
            q = q->next;
        }

        if (q == first)
        {
            first = last = NULL;
        }
        else
        {

            q1->next = NULL;
            last = q1;
        }

        delete q;

        cout << "\nLast node deleted successfully." << endl;
    }
}

void display()
{
    Nodetype* p = first;

    cout << "\nLinked List: ";

    while (p != NULL)
```

```
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

int main()
{
    int n;

    cout << "How many nodes do you want to insert? ";
    cin >> n;

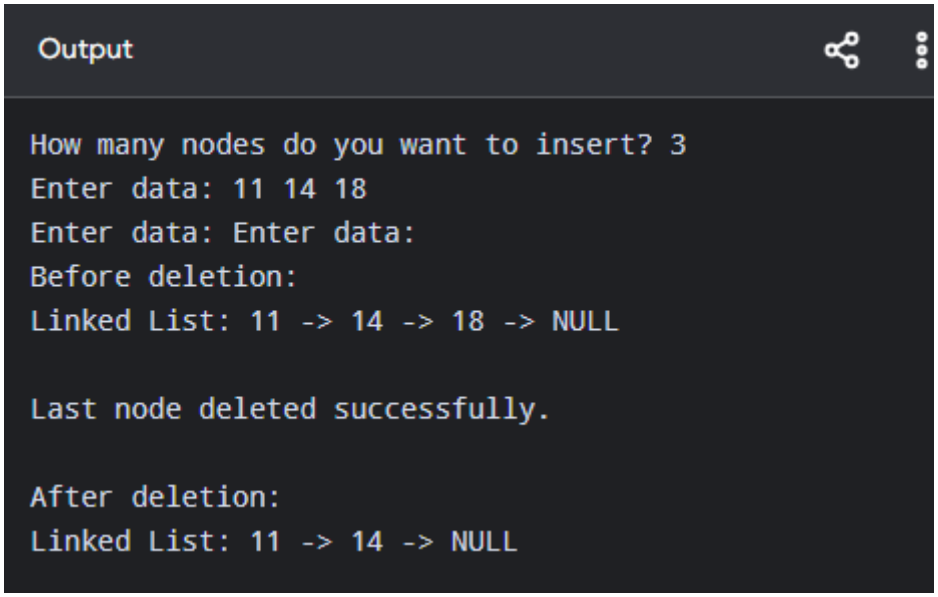
    for (int i = 0; i < n; i++)
    {
        insert_end();
    }

    cout << "\nBefore deletion:";
    display();

    delete_last();

    cout << "\nAfter deletion:";
    display();

    return 0;
}
```



ACTIVITY 7

```
#include <iostream>

using namespace std;


struct Nodetype

{

   int data;

   Nodetype* next;

};


Nodetype* first = NULL;

Nodetype* last = NULL;


void insert_end()

{

   Nodetype* p = new Nodetype;


   cout << "Enter data: ";
```

```
    cin >> p->data;

    p->next = NULL;

    if (first == NULL)
    {
       first = last = p;
    }
    else
    {
       last->next = p;
       last = p;
    }
}

void remove_spec(int  key)
{
    Nodetype* q;
    Nodetype* q1;

    q1 = NULL;
    q = first;

    while (q != NULL && q->data != key)
    {
       q1 = q;
       q = q->next;
    }

    if (q == NULL)
    {
```

```
    cout << "Not found supplied  key." << endl;
}

else if (q == first && q == last)
{
    delete q;

    first = last = NULL;

    cout << "Node deleted successfully." << endl;
}

else if (q == first)
{
    first = first->next;

    delete q;

    cout << "Node deleted successfully." << endl;
}

else if (q == last)
{
    q1->next = NULL;

    last = q1;

    delete q;

    cout << "Node deleted successfully." << endl;
}
```

```
    else
    {
        q1->next = q->next;

        delete q;

        cout << "Node deleted successfully." << endl;
    }
}

void display()
{
    Nodetype* p = first;

    cout << "\nLinked List: ";

    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

int main()
{
    int n;
    int key;
```

```
    cout << "How many nodes do you want to insert? ";
    cin >> n;

    for (int i = 0; i < n; i++)
    {
       insert_end();
    }

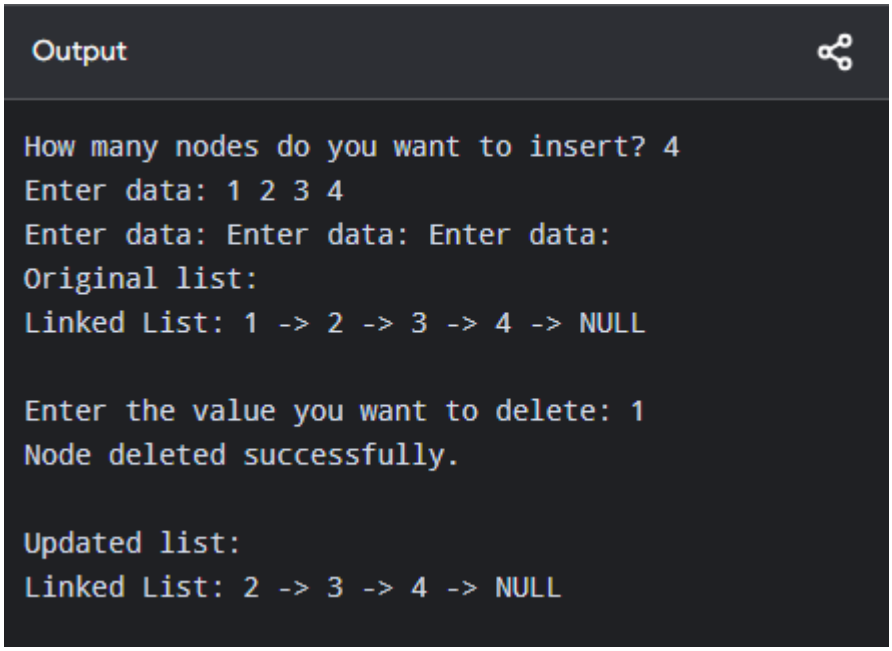
    cout << "\nOriginal list:";
    display();

    cout << "\nEnter the value you want to delete: ";
    cin >> key;

    remove_spec(key);

    cout << "\nUpdated list:";
    display();

    return 0;
}
```



ACTIVITY 8

```
#include <iostream>

using namespace std;


struct Nodetype

{

   int data;

   Nodetype* next;

};


Nodetype* first = NULL;

Nodetype* last = NULL;


void insert_end()

{

   Nodetype* p = new Nodetype;


   cout << "Enter data: ";
```

```
    cin >> p->data;

    p->next = NULL;

    if (first == NULL)
    {
        first = last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
}

void display()
{
    Nodetype* p = first;

    cout << "\nLinked List: ";

    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

void delete_all()
```

```
{
    Nodetype* p;

    while (first != NULL)
    {

        p = first;
        first = first->next;
        delete p;
    }

    last = NULL;

    cout << "\nComplete linked list deleted." << endl;
}

int main()
{
    int n;

    cout << "How many nodes do you want to insert? ";
    cin >> n;

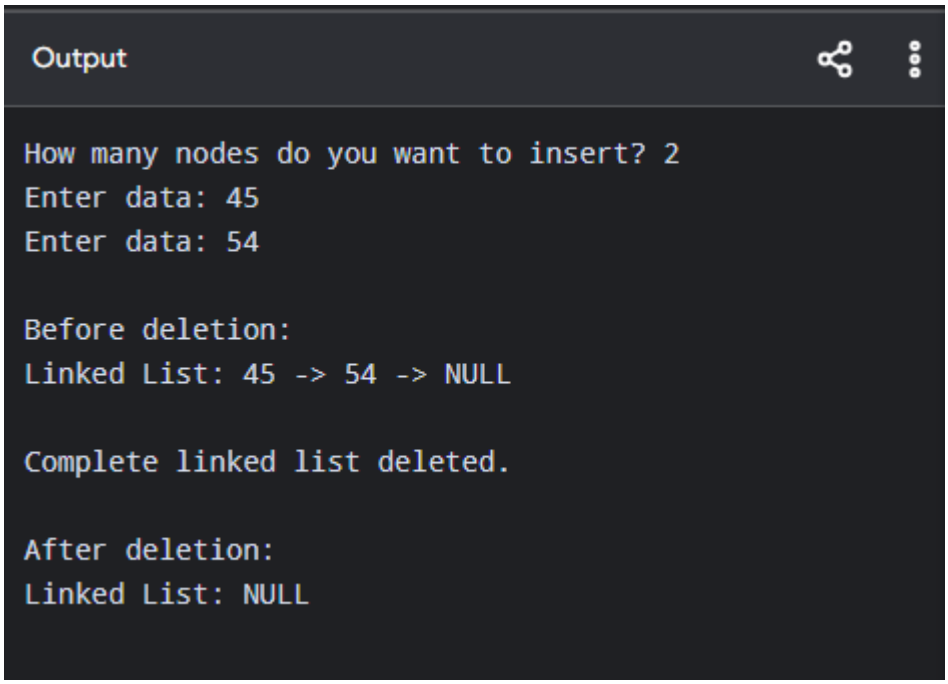
    for (int i = 0; i < n; i++)
    {
        insert_end();
    }

    cout << "\nBefore deletion:";
    display();
```

```
    delete_all();

    cout << "\nAfter deletion:";
    display();

    return 0;
}
```



Lab Task 1:

```
#include <iostream>
using namespace std;

struct node
{
    int data;
    node *next;
};
```

```
void display(node *start)
{
    node *temp = start;

    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

void reverseLoop(node *&start)
{
    node *prev = nullptr;
    node *current = start;
    node *next = nullptr;

    while (current != nullptr)
    {

        next = current->next;

        current->next = prev;


        prev = current;
```

```
        current = next;
    }

    start = prev;
}

int main()
{
    node *start = nullptr;


    node *newnode = new node;
    newnode->data = 10;
    newnode->next = nullptr;
    start = newnode;

    newnode = new node;
    newnode->data = 20;
    newnode->next = nullptr;
    start->next = newnode;

    newnode = new node;
    newnode->data = 30;
    newnode->next = nullptr;
    start->next->next = newnode;

    cout << "Before Reverse:" << endl;
    display(start);
```

```
    reverseLoop(start);

    cout << "After Reverse:" << endl;
    display(start);

    return 0;
}
```

```
Before Reverse:
10 -> 20 -> 30 -> NULL
After Reverse:
30 -> 20 -> 10 -> NULL
PS C:\Users\FA25-BSE-064\Desktop\lab2>
```

Lab Task 2:

```
#include <iostream>
using namespace std;

struct node
{
    int data;
    node *next;
};

void insert(node *&start, int data)
{
    node *newnode = new node;
    newnode->data = data;
    newnode->next = nullptr;
```

```
    if (start == nullptr)
    {
        start = newnode;
        return;
    }

    node *temp = start;

    while (temp->next  != nullptr)
    {
        temp = temp->next;
    }

    temp->next = newnode;
}

void display(node  *start)
{
    node *temp = start;

    while (temp != nullptr)
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

node* mergeLists(node  *start1, node *start2)
```

```
{
    node *start3 = nullptr;

    node *temp = start1;

    while (temp != nullptr)
    {
        insert(start3, temp->data);
        temp = temp->next;
    }

    temp = start2;

    while (temp != nullptr)
    {
        insert(start3, temp->data);
        temp = temp->next;
    }

    return start3;
}

int main()
{
    node *start1 = nullptr;
    node *start2 = nullptr;
    node *start3 = nullptr;

    // First linked list
    insert(start1, 10);
    insert(start1, 20);
```

```
    insert(start1, 30);

    // Second linked list
    insert(start2, 40);
    insert(start2, 50);
    insert(start2, 60);

    cout << "First Linked List:" << endl;
    display(start1);

    cout << "Second Linked List:" << endl;
    display(start2);

   start3 = mergeLists(start1, start2);

    cout << "Third Linked List after merging:" << endl;
    display(start3);

    return 0;
}
```

Lab Task 3:

```
#include <iostream>
using namespace std;

struct node
{
    int data;
    node *next;
```

```
};

void insert(node *&start, int data)
{
    node *newnode = new node;
    newnode->data = data;
    newnode->next = nullptr;

    if (start == nullptr)
    {
        start = newnode;
        return;
    }

    node *temp = start;

    while (temp->next != nullptr)
    {
        temp = temp->next;
    }

    temp->next = newnode;
}

void display(node *start)
{
    node *temp = start;

    while (temp != nullptr)
```

```
    {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

void findMultiple(node *start, int value)
{
    node *temp = start;
    int count = 0;

    while (temp != nullptr)
    {
        if (temp->data == value)
        {
            count++;
        }

        temp = temp->next;
    }

    if (count > 1)
    {
        cout << value << " occurs " << count << " times." << endl;
    }
    else if (count == 1)
    {
```

```
        cout << value << " occurs only once." << endl;
    }
    else
    {
        cout << value << " does not exist in the list." << endl;
    }
}

int main()
{
    node *start = nullptr;

    insert(start, 10);
    insert(start, 20);
    insert(start, 10);
    insert(start, 30);
    insert(start, 10);
    insert(start, 40);

    cout << "Linked List:" << endl;
    display(start);

    int value;
    cout << "Enter data to find: ";
    cin >> value;

    findMultiple(start, value)
    return 0;
}
```

SINGLY LINKED LIST ADDITIONAL

Task 1:

```
#include <iostream>
using namespace std;
struct Node {
   int data;
   Node* next;
   Node(int value) {
      data = value;
      next = nullptr;
   }
};
void removeDuplicates(Node*  head) {
  Node* current = head;
   while (current != nullptr && current->next != nullptr) {
      if (current->data == current->next->data) {
         Node* duplicate = current->next;
         current->next = current->next->next;
         delete duplicate;
      } else {
         current = current->next;
      }
   }
}
void display(Node*  head) {
   Node* temp = head;
   while (temp != nullptr) {
      cout << temp->data;

      if (temp->next != nullptr)
```

```
            cout << " -> ";
        temp = temp->next;
    }
    cout << endl;
}
int main() {
    Node* head = new Node (10);
    Node* second = new Node (20);
    Node* third = new Node (20);
    Node* forth = new Node (30);
    Node* fifth = new Node (30);
    head->next=second;
    second->next=third;
    third->next=forth;
    forth->next=fifth;
    cout << "Original list: ";
    display(head);
    removeDuplicates(head);

    cout << "After removing duplicates: ";
    display(head);

    return 0;
}
```

Output

```
Original list: 10 -> 20 -> 20 -> 30 -> 30
After removing duplicates: 10 -> 20 -> 30
```

Task 2:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = nullptr;
    }
};
bool hasCycle(Node* head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;          // Move 1 step
        fast = fast->next->next;    // Move 2 steps

        if (slow == fast) {
            return true;            // Cycle detected
        }
    }

    return false;                   // No cycle
}
void display(Node* head) {
    Node* temp = head;

    while (temp != nullptr) {
```

```
        cout << temp->data;

        if (temp->next  != nullptr)
            cout << " -> ";

        temp = temp->next;
    }

    cout << endl;
}

int main() {
    Node* head = new Node (10);
    Node* second = new Node (20);
    Node* third = new Node (20);
    Node* forth = new Node (30);
    Node* fifth = new Node (30);
    head->next=second;
    second->next=third;
    third->next=forth;
    forth->next=fifth;

    cout << "list: ";
    display(head);

    if(hasCycle(head)){
        cout<<"yes the list contain a circular loop";
    }
    else{
        cout<<"no, the lest is not circular";
    }
```

```
    return 0;
}
```

```
list: 10 -> 20 -> 20 -> 30 -> 30
no, the lest is not circular
```

Task 3:

```
#include <iostream>
using namespace std;
struct Node {
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = nullptr;
    }
};

int countElement(Node*  head) {
    Node* curr = head;
    int count = 1;

    while (curr->next != nullptr) {
        curr = curr->next;
        count++;
    }
```

```
    return count;
}

int middleElement(Node*  head) {
    Node* slow = head;
    Node* fast = head;

    while (fast != nullptr  && fast->next != nullptr)  {
        slow = slow->next;
        fast = fast->next->next;
    }

    return slow->data;
}

void display(Node*  head) {
    Node* temp = head;

    for (int i = 0; i < 5 && temp != nullptr;  i++) {
        cout << temp->data;

        if (i < 4)
            cout << " -> ";

        temp = temp->next;
    }

    cout << endl;
}

int main() {
```

```
    Node* head = new Node(10);
    Node* second = new Node(20);
    Node* third = new Node(30);
    Node* fourth = new Node(40);
    Node* fifth = new Node(50);

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;

    cout << "List: ";
    display(head);

    cout << "Number of elements in list: "
        << countElement(head) << endl;

    cout << "Middle element: "
        << middleElement(head) << endl;

    return 0;
}
```

Output

```
List: 10 -> 20 -> 30 -> 40 -> 50
Number of elements in list: 5
Middle element: 30
```

Task 4:

```
#include <iostream>
using namespace std;

{
   int data;
   Nodetype* next;
};

void insert_end(Nodetype*&  first, Nodetype*& last, int value)
{
   Nodetype* p = new Nodetype;

   p->data = value;
   p->next = NULL;

   if (first == NULL)
   {
      first = last = p;
   }
   else
   {
      last->next = p;
      last = p;
   }
}

void display(Nodetype*  first)
{
   Nodetype* p = first;
```

```
    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

Nodetype* reverse_k_groups(Nodetype*  first, int k)
{

    if (first == NULL || k <= 1)
    {
        return first;
    }

    Nodetype* temp = first;

    for (int i = 0; i < k; i++)
    {
        if (temp == NULL)
        {
            return first;
        }

        temp = temp->next;
    }

    Nodetype* previous = NULL;
    Nodetype* current = first;
```

```
    Nodetype* next = NULL;

    int count = 0;

    while (current  != NULL && count < k)
    {
        next = current->next;
        current->next  = previous;
        previous = current;
        current = next;
        count++;
    }

    first->next = reverse_k_groups(current,  k);
    return previous;
}


int main()
{
    Nodetype* first = NULL;
    Nodetype* last = NULL;

    insert_end(first,  last, 1);
    insert_end(first,  last, 2);
    insert_end(first,  last, 3);
    insert_end(first,  last, 4);
    insert_end(first,  last, 5);
    insert_end(first,  last, 6);

    cout << "Original List: ";
```

```
    display(first);

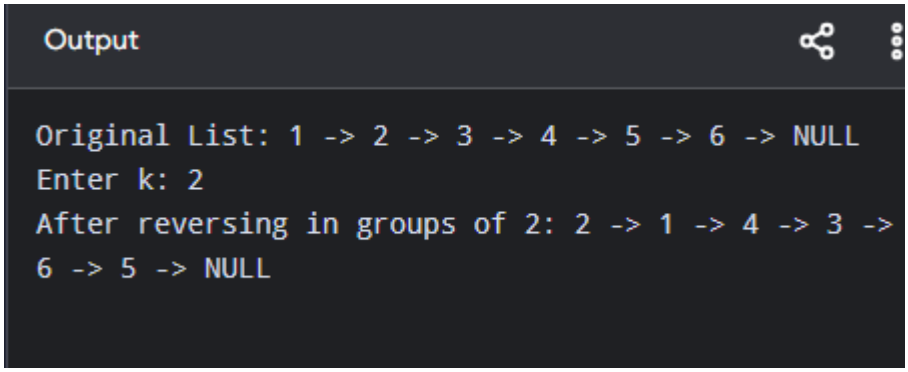
    int k;

    cout << "Enter k: ";
    cin >> k;

    first = reverse_k_groups(first,  k);

    cout << "After reversing in groups of " << k << ": ";
    display(first);

    return 0;
}
```



Task 5:

```
#include <iostream>
using namespace std;

// Node structure
struct Nodetype
{
    int data;
```

```
    Nodetype* next;
};

// Function to insert a node at the end
void insert_end(Nodetype*& first, Nodetype*& last, int value)
{
    Nodetype* p = new Nodetype;

    p->data = value;
    p->next = NULL;

    if (first == NULL)
    {
        first = last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
}

// Function to display a linked list
void display(Nodetype* first)
{
    Nodetype* p = first;

    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
```

```
    }

    cout << "NULL" << endl;
}

// Function to merge two sorted linked lists
Nodetype* merge_lists(Nodetype* first1, Nodetype* first2)
{
    // If first list is empty
    if (first1 == NULL)
        return first2;

    // If second list is empty
    if (first2 == NULL)
        return first1;

    Nodetype* result = NULL;

    // Compare the first nodes
    if (first1->data <= first2->data)
    {
        result = first1;

        // Recursively merge remaining nodes
        result->next = merge_lists(first1->next, first2);
    }
    else
    {
        result = first2;

        // Recursively merge remaining nodes
```

```
        result->next = merge_lists(first1, first2->next);
    }

    return result;
}

int main()
{
    Nodetype* first1 = NULL;
    Nodetype* last1 = NULL;

    Nodetype* first2 = NULL;
    Nodetype* last2 = NULL;

    // List 1
    insert_end(first1, last1, 1);
    insert_end(first1, last1, 3);
    insert_end(first1, last1, 5);

    // List 2
    insert_end(first2, last2, 2);
    insert_end(first2, last2, 4);
    insert_end(first2, last2, 6);

    cout << "List 1: ";
    display(first1);

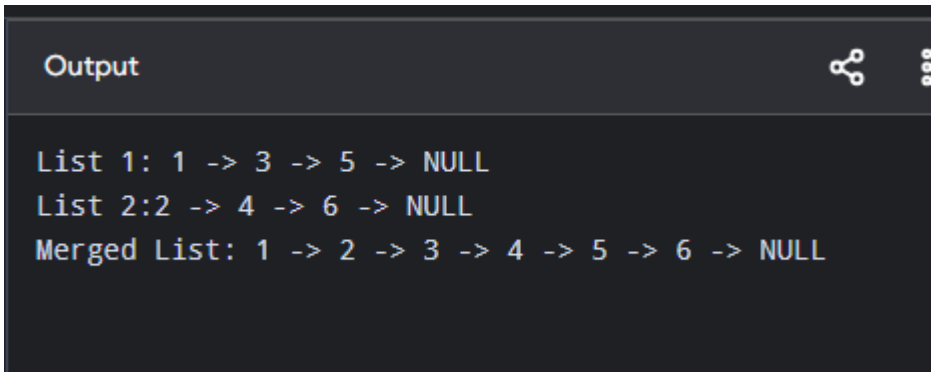
    cout << "List 2: ";
    display(first2);

    // Merge the two lists
```

```
    Nodetype* merged = merge_lists(first1, first2);

    cout << "Merged List: ";
    display(merged);

    return 0;
}
```



Task 6:

```
#include <iostream>
using namespace std;

// Node structure
struct Nodetype
{
    int data;
    Nodetype* next;
};

// Insert node at the end
void insert_end(Nodetype*& first, Nodetype*& last, int value)
{
    Nodetype* p = new Nodetype;
```

```
    p->data = value;
    p->next = NULL;

    if (first == NULL)
    {
        first = last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
}

// Display linked list
void display(Nodetype*  first)
{
    Nodetype* p = first;

    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

// Find Nth node from the end
Nodetype* nth_from_end(Nodetype*  first, int n)
{
```

```
    Nodetype* p = first;
    Nodetype* q = first;

    // Move p n steps ahead
    for (int i = 0; i < n; i++)
    {
        if (p == NULL)
        {
            return NULL;
        }

        p = p->next;
    }

    // Move both pointers together
    while (p != NULL)
    {
        p = p->next;
        q = q->next;
    }

    return q;
}

int main()
{
    Nodetype* first = NULL;
    Nodetype* last = NULL;

    int n;
```

```
    // Create the list
    insert_end(first, last, 1);
    insert_end(first, last, 2);
    insert_end(first, last, 3);
    insert_end(first, last, 4);
    insert_end(first, last, 5);

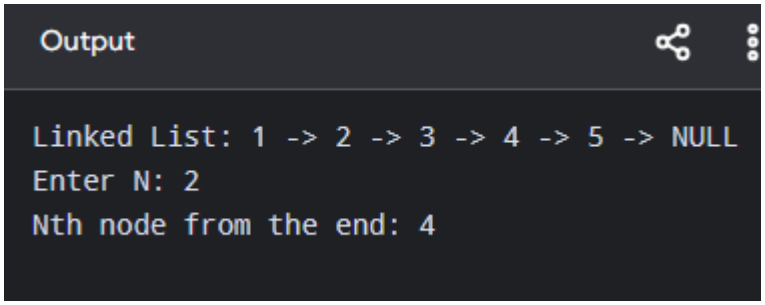
    cout << "Linked List: ";
    display(first);

    // Ask for N
    cout << "Enter N: ";
    cin >> n;

    // Find Nth node from end
    Nodetype* result = nth_from_end(first, n);

    if (result == NULL)
    {
        cout << "Invalid value of N." << endl;
    }
    else
    {
        cout << "Nth node from the end: "
             << result->data << endl;
    }

    return 0;
}
```



Task 7:

```
#include <iostream>

using namespace std;


struct Nodetype

{

   int data;

   Nodetype* next;

};


void insert_end(Nodetype*&  first, Nodetype*& last, int value)

{

   Nodetype* p = new Nodetype;


   p->data = value;

   p->next = NULL;


   if (first == NULL)

   {

      first = last = p;

   }

   else

   {

      last->next = p;
```

```
        last = p;
    }
}

void display(Nodetype*  first)
{
    Nodetype* p = first;

    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

Nodetype* reverse_list(Nodetype*  head)
{
    Nodetype* previous = NULL;
    Nodetype* current = head;
    Nodetype* next = NULL;

    while (current  != NULL)
    {
        next = current->next;
        current->next  = previous;
        previous = current;
        current = next;
    }
```

```
    return previous;
}

bool is_palindrome(Nodetype*  first)
{
    if (first == NULL || first->next == NULL)
    {
        return true;
    }

    Nodetype* slow = first;
    Nodetype* fast = first;

    while (fast != NULL && fast->next != NULL)
    {
        slow = slow->next;
        fast = fast->next->next;
    }
    if (fast != NULL)
    {
        slow = slow->next;
    }

    Nodetype* second_half = reverse_list(slow);

    Nodetype* first_half = first;
    Nodetype* temp = second_half;

    while (temp != NULL)
    {
        if (first_half->data  != temp->data)
```

```
        {
            return false;
        }

        first_half = first_half->next;
        temp = temp->next;
    }

    return true;
}

int main()
{
    Nodetype* first = NULL;
    Nodetype* last = NULL;

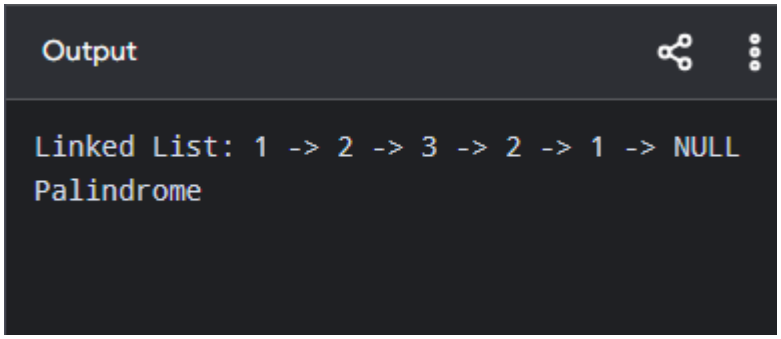
    insert_end(first, last, 1);
    insert_end(first, last, 2);
    insert_end(first, last, 3);
    insert_end(first, last, 2);
    insert_end(first, last, 1);

    cout << "Linked List: ";
    display(first);

    if (is_palindrome(first))
    {
        cout << "Palindrome" << endl;
    }
    else
    {
```

```
        cout << "Not Palindrome" << endl;
    }

    return 0;
}
```



Task 8:

```
#include <iostream>
using namespace std;

struct Nodetype
{
    int data;
    Nodetype* next;
};

void insert_end(Nodetype*&  first, Nodetype*& last, int value)
{
    Nodetype* p = new Nodetype;

    p->data = value;
    p->next = NULL;

    if (first == NULL)
    {
```

```
        first = last = p;
    }
    else
    {
        last->next = p;
        last = p;
    }
}

void display(Nodetype*  first)
{
    Nodetype* p = first;

    while (p != NULL)
    {
        cout << p->data << " -> ";
        p = p->next;
    }

    cout << "NULL" << endl;
}

Nodetype* rotate_right(Nodetype*  first, int k)
{

    if (first == NULL || first->next == NULL)
    {
        return  first;
    }

    int length = 1;
```

```
Nodetype* last = first;

while (last->next != NULL)
{
   last = last->next;
   length++;
}

k = k % length;

if (k == 0)
{
   return first;
}

last->next = first;

int steps = length - k;

Nodetype* new_last = first;

for (int i = 1; i < steps; i++)
{
   new_last = new_last->next;
}

Nodetype* new_first = new_last->next;

new_last->next = NULL;

return new_first;
```

```
}


int main()
{
    Nodetype* first = NULL;
    Nodetype* last = NULL;

    insert_end(first, last, 1);
    insert_end(first, last, 2);
    insert_end(first, last, 3);
    insert_end(first, last, 4);
    insert_end(first, last, 5);

    cout << "Original List: ";
    display(first);

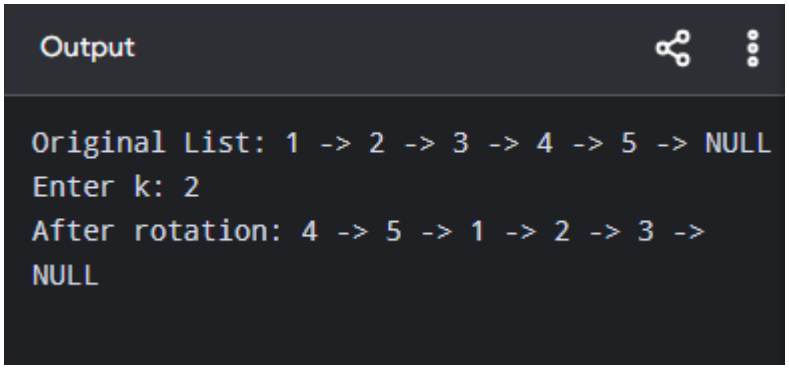
    int k;

    cout << "Enter k: ";
    cin >> k;

    first = rotate_right(first, k);

    cout << "After rotation: ";
    display(first);

    return 0;
}
```


Output
Original List: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
Enter k: 2
After rotation: 4 -> 5 -> 1 -> 2 -> 3 ->
NULL

LAB 3

DOUBLY LINKED LIST

ACTIVITY 1

```
#include <iostream>
using namespace std;

struct Nodetype
{
   int data;
   Nodetype *prev;
   Nodetype *next;
};

Nodetype *first = NULL;
Nodetype *last = NULL;

void insert_end()
{
   Nodetype *p;

   p = new Nodetype;

   cout << "Enter the data in node: ";
   cin >> p->data;

   p->next = NULL;
   p->prev = NULL;

   if (first == NULL)
   {
```

```
        first = last = p;
    }
    else
    {
        last->next = p;
        p->prev = last;
        last = p;
    }
}

void display()
{
    Nodetype *p = first;

    cout << "Doubly Linked List: ";

    while (p != NULL)
    {
        cout << p->data << " ";
        p = p->next;
    }

    cout << endl;
}

int main()
{
    int n;

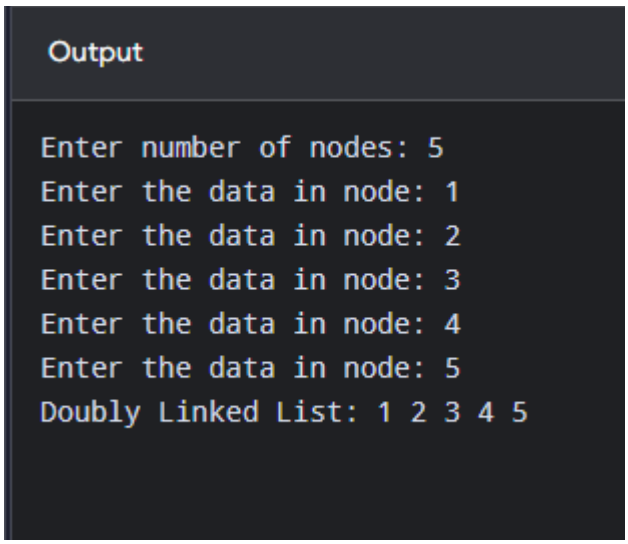
    cout << "Enter number of nodes: ";
    cin >> n;
```

```
    for (int i = 0; i < n; i++)
    {
        insert_end();
    }

    display();

    return 0;
}
```

OUTPUT:



ACTIVITY 2

```
#include <iostream>

using namespace std;


struct Node

{

   int data;

   Node *prev, *next;

};


Node *first = NULL, *last = NULL;


void displayForward()

{

   Node *p;

   for (p = first; p != NULL; p = p->next)

      cout << p->data << " ";

}


void displayBackward()

{

   Node *p;

   for (p = last; p != NULL; p = p->prev)

      cout << p->data << " ";

}


int main()

{

   Node *a = new Node{10, NULL, NULL};
```

```
    Node *b = new Node{20, a, NULL};
    Node *c = new Node{30, b, NULL};

    a->next = b;
    b->next = c;

    first = a;
    last = c;

    cout << "Forward: ";
    displayForward();

    cout << "\nBackward: ";
    displayBackward();

    return 0;
}
```

Output

```
Forward: 10 20 30
Backward: 30 20 10
```

ACTIVITY 3

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *prev, *next;
};

Node *first = NULL, *last = NULL;

void insert_begin(Node  *p)
{
   if (first == NULL)
      first = last = p, p->next = p->prev = NULL;
   else
   {
      p->next = first;
      p->prev = NULL;
      first->prev = p;
      first = p;
   }
}

void insert_end(Node  *p)
{
   if (first == NULL)
      first = last = p, p->next = p->prev = NULL;
   else
   {
```

```
        p->next = NULL;
        p->prev = last;
        last->next = p;
        last = p;
    }
}

void insert_after(Node *q, Node *p)
{
    p->prev = q;
    p->next = q->next;

    if (q->next != NULL)
        q->next->prev = p;

    q->next = p;

    if (last == q)
        last = p;
}

void display()
{
    for (Node *p = first; p != NULL; p = p->next)
        cout << p->data << " ";
}

int main()
{
    Node *p;
```

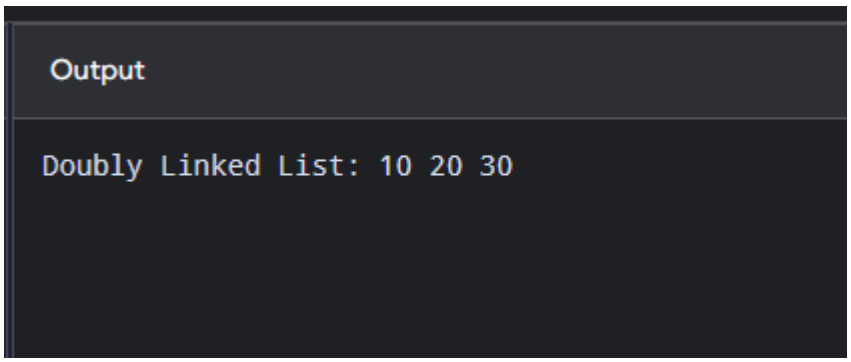
```
    // Insert at beginning
    p = new Node{10, NULL, NULL};
    insert_begin(p);

    // Insert at end
    p = new Node{30, NULL, NULL};
    insert_end(p);

    // Insert after node 10
    p = new Node{20, NULL, NULL};
    insert_after(first, p);

    cout << "Doubly Linked List: ";
    display();

    return 0;
}
```



ACTIVITY 4

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *prev, *next;
};

Node *first = NULL, *last = NULL;

void delete_first()
{
   if (first == NULL) return;

   Node *p = first;
   first = first->next;

   if (first == NULL)
      last = NULL;
   else
      first->prev = NULL;

   delete p;
}

void delete_last()
{
   if (last == NULL) return;
```

```
    Node *p = last;
    last = last->prev;

    if (last == NULL)
        first = NULL;
    else
        last->next = NULL;

    delete p;
}

void delete_key(int  key)
{
    Node *p = first;

    while (p != NULL && p->data != key)
        p = p->next;

    if (p == NULL) return;  // key not found

    if (p == first && p == last)
    {
        first = last = NULL;
    }
    else if (p == first)
    {
        first = first->next;
        first->prev = NULL;
    }
    else if (p == last)
    {
```

```
        last = last->prev;
        last->next = NULL;
    }
    else
    {
        p->prev->next = p->next;
        p->next->prev = p->prev;
    }

    delete p;
}

void display()
{
    for (Node *p = first; p != NULL; p = p->next)
        cout << p->data << " ";
}

int main()
{
    // Create list: 10 <-> 20 <-> 30 <-> 40
    Node *a = new Node{10, NULL, NULL};
    Node *b = new Node{20, a, NULL};
    Node *c = new Node{30, b, NULL};
    Node *d = new Node{40, c, NULL};

    a->next = b;
    b->next = c;
    c->next = d;

    first = a;
```

```
    last = d;

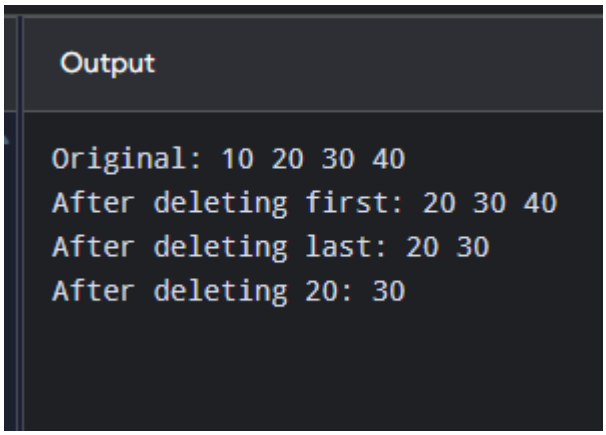
    cout << "Original: ";
    display();

    delete_first();
    cout << "\nAfter deleting first: ";
    display();

    delete_last();
    cout << "\nAfter deleting last: ";
    display();

    delete_key(20);
    cout << "\nAfter deleting 20: ";
    display();

    return 0;
}
```



ACTIVITY 5

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *prev, *next;
};

Node *first = NULL, *last = NULL;

void delete_list()
{
   Node *p;

   while (first != NULL)
   {
      p = first;
      first = first->next;
      delete p;
   }

   last = NULL;
}

void display()
{
   for (Node *p = first; p != NULL; p = p->next)
      cout << p->data << " ";
}
```

```
int main()
{
    // Create list: 10 <-> 20 <-> 30
    Node *a = new Node{10, NULL, NULL};
    Node *b = new Node{20, a, NULL};
    Node *c = new Node{30, b, NULL};

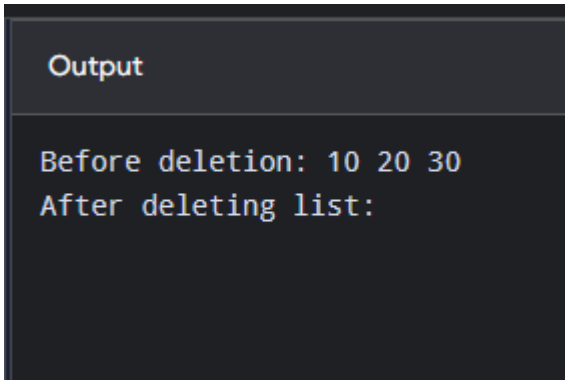
    a->next = b;
    b->next = c;

    first = a;
    last = c;

    cout << "Before deletion: ";
    display();

    delete_list();

    cout << "\nAfter deleting list: ";
    display();
    return 0;
}
```



LAB TASK 1

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *prev, *next;
};

Node *first = NULL, *last = NULL;

void reverse()
{
   Node *p = first;

   while (p != NULL)
   {
      Node *temp = p->next;
      p->next = p->prev;
      p->prev = temp;
      p = temp;
   }

   Node *temp = first;
   first = last;
   last = temp;
}

void display()
{
```

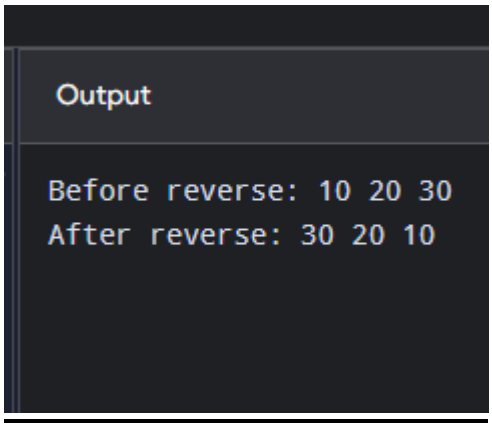
```
    for (Node *p = first; p != NULL; p = p->next)
        cout << p->data << " ";
}

int main()
{
    Node *a = new Node{10, NULL, NULL};
    Node *b = new Node{20, a, NULL};
    Node *c = new Node{30, b, NULL};

    a->next = b;
    b->next = c;

    first = a;
    last = c;

    cout << "Before reverse: ";
    display();
    reverse();
    cout << "\nAfter reverse: ";
    display();
    return 0;
}
```



LAB TASK 2

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *prev, *next;
};

Node *first = NULL, *last = NULL;

void swapNodes(int x, int y)
{
   Node *p = first, *q = first;

   while (p != NULL && p->data != x)
      p = p->next;

   while (q != NULL && q->data != y)
      q = q->next;

   if (p == NULL || q == NULL || p == q)
   {
      cout << "Node not found!";
      return;
   }

   // Adjacent: p before q
   if (p->next == q)
```

```
{
   Node *a = p->prev, *b = q->next;

   if (a) a->next = q; else first = q;
   if (b) b->prev = p; else last = p;

   q->prev = a;
   q->next = p;
   p->prev = q;
   p->next = b;
}
// Adjacent: q before p
else if (q->next == p)
{
   swapNodes(y, x);
}
// Non-adjacent
else
{
   Node *pp = p->prev, *pn = p->next;
   Node *qp = q->prev, *qn = q->next;

   if (pp) pp->next = q; else first = q;
   if (pn) pn->prev = q;

   if (qp) qp->next = p; else first = p;
   if (qn) qn->prev = p;

   p->prev = qp;
   p->next = qn;
   q->prev = pp;
```

```
        q->next = pn;

        if (last == p) last = q;
        else if (last == q) last = p;
    }
}

void display()
{
    for (Node *p = first; p != NULL; p = p->next)
        cout << p->data << " ";
}

int main()
{
    Node *a = new Node{10, NULL, NULL};
    Node *b = new Node{20, a, NULL};
    Node *c = new Node{30, b, NULL};
    Node *d = new Node{40, c, NULL};

    a->next = b;
    b->next = c;
    c->next = d;

    first = a;
    last = d;

    cout << "Original list: ";
    display();

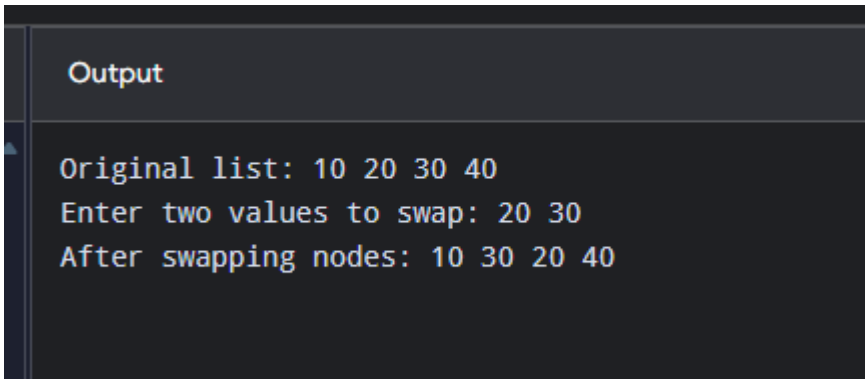
    int x, y;
```

```
    cout << "\nEnter two values to swap: ";
    cin >> x >> y;

    swapNodes(x, y);

    cout << "After swapping nodes: ";
    display();

    return 0;
}
```



LAB TASK 3

```
#include <iostream>
using namespace std;

struct SNode
{
    int data;
    SNode *next;
};

struct DNode
{
    int data;
```

```
    DNode *prev, *next;
};

DNode* convert(SNode *first)
{
    DNode *dFirst = NULL, *last = NULL;

    while (first != NULL)
    {
        DNode *p = new DNode{first->data, last, NULL};

        if (dFirst == NULL)
            dFirst = p;
        else
            last->next = p;

        last = p;
        first = first->next;
    }

    return dFirst;
}

void display(SNode *p)
{
    while (p != NULL)
    {
        cout << p->data << " ";
        p = p->next;
    }
}
```

```
void displayD(DNode *p)
{
   while (p != NULL)
   {
      cout << p->data << " ";
      p = p->next;
   }
}

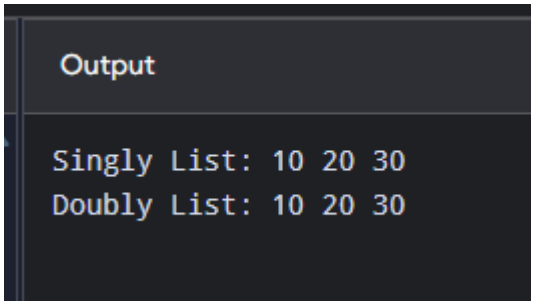
int main()
{
   // Singly linked list: 10 -> 20 -> 30
   SNode *first = new SNode{10, NULL};
   first->next = new SNode{20, NULL};
   first->next->next = new SNode{30, NULL};

   cout << "Singly List: ";
   display(first);

   DNode *dFirst = convert(first);

   cout << "\nDoubly List: ";
   displayD(dFirst);

   return 0;
}
```


Output
Singly List: 10 20 30
Doubly List: 10 20 30

DOUBLY LINKED LIST ADDITIONAL

TASK 1

```
#include <iostream>
#include <string>
using namespace std;

struct Node
{
   string title, artist;
   Node *prev, *next;
};

class Playlist
{
   Node *first, *last, *current;

public:
   Playlist()
   {
      first = last = current = NULL;
   }

   void addSong(string  title, string artist)
   {
      Node *p = new Node{title, artist, last, NULL};

      if (first == NULL)
         first = last = p;
      else
```

```
    {
        last->next = p;
        last = p;
    }

    if (current == NULL)
        current = first;
}

void playNext()
{
    if (current != NULL && current->next != NULL)
        current = current->next;
}

void playPrevious()
{
    if (current != NULL && current->prev != NULL)
        current = current->prev;
}

void removeCurrentSong(Node *p)
{
    if (p == NULL) return;

    Node *next = p->next;
    Node *prev = p->prev;

    if (prev)
        prev->next = next;
    else
```

```
            first = next;

        if (next)
            next->prev = prev;
        else
            last = prev;

        // Move playback pointer
        if (next)
            current = next;
        else
            current = prev;

        delete p;
    }

    Node* getCurrent()
    {
        return current;
    }

    void showCurrent()
    {
        if (current)
            cout << "Playing: " << current->title
                 << " - " << current->artist << endl;
        else
            cout << "Playlist is empty\n";
    }
};
```

```
int main()
{
    Playlist p;

    p.addSong("Song A", "Artist A");
    p.addSong("Song B", "Artist B");
    p.addSong("Song C", "Artist C");

    p.showCurrent();

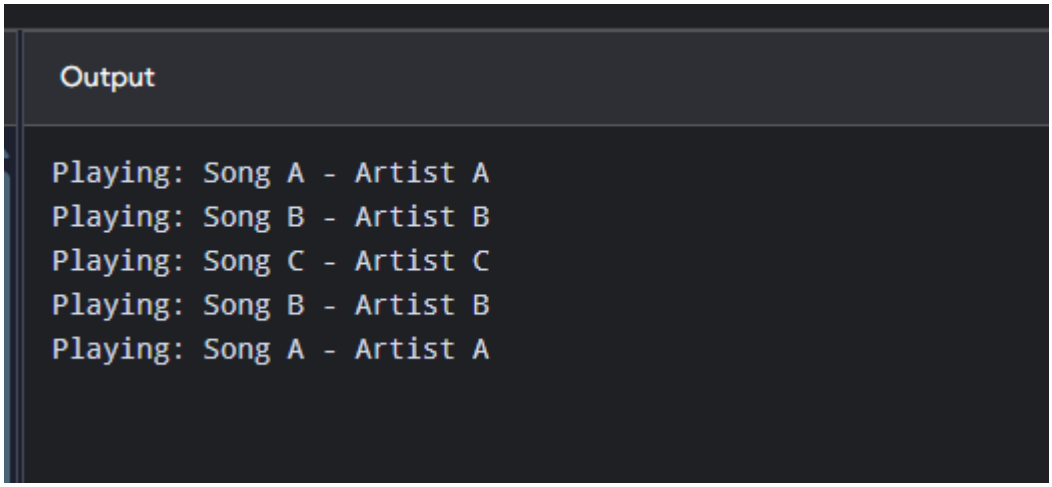
    p.playNext();
    p.showCurrent();

    p.playNext();
    p.showCurrent();

    // Remove currently playing song
    p.removeCurrentSong(p.getCurrent());
    p.showCurrent();

    p.playPrevious();
    p.showCurrent();

    return 0;
}
```



TASK 2

```
#include <iostream>
#include <string>
using namespace std;

struct Node
{
   string url;
   Node *prev, *next;
};

class Browser
{
   Node *currentPage;

public:
   Browser()
   {
      currentPage = NULL;
   }
```

```
void visitPage(string url)
{
   Node *p = new Node{url, NULL, NULL};

   if (currentPage == NULL)
   {
      currentPage = p;
      return;
   }

   // Delete forward history
   Node *temp = currentPage->next;
   while (temp != NULL)
   {
      Node *del = temp;
      temp = temp->next;
      delete del;
   }

   currentPage->next = p;
   p->prev = currentPage;
   currentPage = p;
}

void goBack()
{
   if (currentPage != NULL && currentPage->prev != NULL)
      currentPage = currentPage->prev;
}

void goForward()
```

```
    {
        if (currentPage != NULL && currentPage->next != NULL)
            currentPage = currentPage->next;
    }

    void showCurrent()
    {
        if (currentPage)
            cout << "Current Page: " << currentPage->url << endl;
        else
            cout << "No page visited.\n";
    }
};

int main()
{
    Browser browser;

    browser.visitPage("google.com");
    browser.visitPage("youtube.com");
    browser.visitPage("github.com");

    browser.showCurrent();

    browser.goBack();
    browser.showCurrent();

    browser.goBack();
    browser.showCurrent();

    browser.goForward();
```

```
    browser.showCurrent();

    // Visiting a new page clears forward history
    browser.visitPage("wikipedia.org");
    browser.showCurrent();

    browser.goForward();    // Nothing happens
    browser.showCurrent();

    return 0;
}
```

Output

```
Current Page: github.com
Current Page: youtube.com
Current Page: google.com
Current Page: youtube.com
Current Page: wikipedia.org
Current Page: wikipedia.org
```

TASK 3

```
#include <iostream>
#include <string>
using namespace std;

struct Node {
    string appName;
    Node* prev;
    Node* next;

    Node(string name) {
        appName = name;
        prev = nullptr;
        next = nullptr;
    }
};

class DoublyLinkedList {
private:
    Node* head;
    Node* tail;

public:
    DoublyLinkedList() {
        head = nullptr;
        tail = nullptr;
    }

    // Add an app to the end of the list
    void addApp(string appName) {
        Node* newNode = new Node(appName);
```

```
    if (head == nullptr) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
}

// Bring an existing app to the front
void bringToFront(string appName) {
    Node* current = head;

    // Search for the app
    while (current != nullptr && current->appName != appName) {
        current = current->next;
    }

    // App not found
    if (current == nullptr) {
        cout << "App not found: " << appName << endl;
        return;
    }

    // Already at the front
    if (current == head) {
        return;
    }

    // Detach current node
```

```
    if (current->prev != nullptr) {
        current->prev->next = current->next;
    }

    if (current->next != nullptr) {
        current->next->prev = current->prev;
    } else {
        // Current node was the tail
        tail = current->prev;
    }

    // Place current node at the head
    current->prev = nullptr;
    current->next = head;

    head->prev = current;
    head = current;
}

// Display apps from head to tail
void display() {
    Node* current = head;

    while (current != nullptr) {
        cout << current->appName;

        if (current->next != nullptr)
            cout << " -> ";

        current = current->next;
    }
```

```
        cout << endl;
    }

    ~DoublyLinkedList() {
        Node* current = head;

        while (current != nullptr) {
            Node* temp = current;
            current = current->next;
            delete temp;
        }
    }
};

int main() {
    DoublyLinkedList apps;

    apps.addApp("WhatsApp");
    apps.addApp("YouTube");
    apps.addApp("Chrome");
    apps.addApp("Instagram");
    apps.addApp("Spotify");

    cout << "Recently used apps: ";
    apps.display();

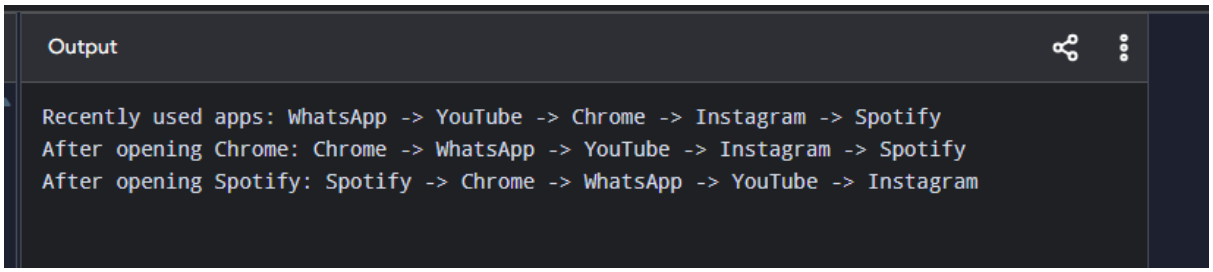
    apps.bringToFront("Chrome");

    cout << "After opening Chrome: ";
    apps.display();
```

```
    apps.bringToFront("Spotify");

    cout << "After opening Spotify: ";
    apps.display();

    return 0;
}
```



TASK 4

```
#include <iostream>
using namespace std;

class TextEditor {
private:
   struct Node {
      char data;
      Node* prev;
      Node* next;

      Node(char c) : data(c), prev(nullptr),  next(nullptr)  {}
   };

   Node* head;
   Node* tail;
   Node* cursor;

public:
   TextEditor() : head(nullptr),  tail(nullptr),  cursor(nullptr)  {}

   // Move cursor one position to the left
   void moveLeft() {
      if (cursor != nullptr)
         cursor = cursor->prev;
   }

   // Move cursor one position to the right
   void moveRight() {
      if (cursor == nullptr)
         return;
```

```
    cursor = cursor->next;
}

// Insert a character before the cursor
void insertAtCursor(char c) {
    Node* newNode = new Node(c);

    // Empty list
    if (head == nullptr) {
        head = tail = newNode;
        cursor = newNode;
        return;
    }

    // Insert before current cursor
    if (cursor != nullptr) {
        newNode->prev = cursor->prev;
        newNode->next = cursor;

        if (cursor->prev != nullptr)
            cursor->prev->next = newNode;
        else
            head = newNode;

        cursor->prev = newNode;
    }
    // Cursor is at the end
    else {
        newNode->prev = tail;
        tail->next = newNode;
```

```
        tail = newNode;
    }
}

// Delete the node at the cursor
void deleteAtCursor() {
    if (cursor == nullptr)
        return;

    Node* temp = cursor;

    // Move cursor to the next node
    cursor = cursor->next;

    // Update previous pointer
    if (temp->prev != nullptr)
        temp->prev->next = temp->next;
    else
        head = temp->next;

    // Update next pointer
    if (temp->next != nullptr)
        temp->next->prev = temp->prev;
    else
        tail = temp->prev;

    delete temp;
}

// Display the text and mark the cursor
void display() {
```

```
        Node* current = head;

        while (current != nullptr) {
            if (current == cursor)
                cout << '|';      // cursor position

            cout << current->data;
            current = current->next;
        }

        // Cursor at the end
        if (cursor == nullptr)
            cout << '|';

        cout << endl;
    }

    ~TextEditor() {
        Node* current = head;

        while (current != nullptr) {
            Node* next = current->next;
            delete current;
            current = next;
        }
    }
};

int main() {
    TextEditor editor;
```

```
    editor.insertAtCursor('H');
    editor.insertAtCursor('e');
    editor.insertAtCursor('l');
    editor.insertAtCursor('l');
    editor.insertAtCursor('o');

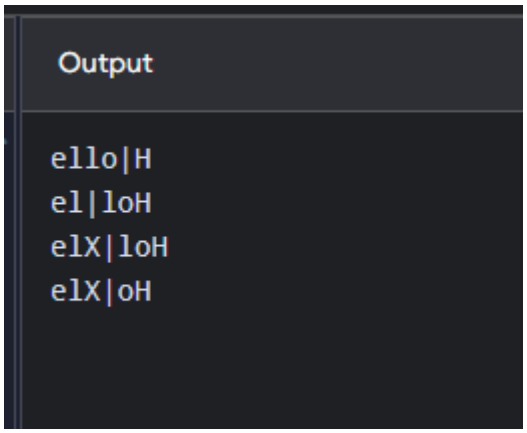
    editor.display();

    editor.moveLeft();
    editor.moveLeft();
    editor.display();

    editor.insertAtCursor('X');
    editor.display();

    editor.deleteAtCursor();
    editor.display();

    return 0;
}
```



TASK 5

```
#include <iostream>
using namespace std;

struct State {
    int value;   // Example canvas data

    State(int v) : value(v) {}
};

struct Node {
    State* state;
    Node* prev;
    Node* next;

    Node(State* s) {
        state = s;
        prev = nullptr;
        next = nullptr;
    }
};

class CanvasHistory {
private:
    Node* head;
    Node* currentState;

public:
    CanvasHistory(State*  initialState) {
        head = new Node(initialState);
        currentState  = head;
```

```
}

// Move one step backward
void undo() {
    if (currentState->prev != nullptr) {
        currentState = currentState->prev;
    }
}

// Move one step forward
void redo() {
    if (currentState->next != nullptr) {
        currentState = currentState->next;
    }
}

// Perform a new action
void performAction(State* newState) {
    // Discard all states ahead of currentState
    Node* temp = currentState->next;

    while (temp != nullptr) {
        Node* nextNode = temp->next;
        delete temp->state;
        delete temp;
        temp = nextNode;
    }

    // The current node is now the last node
    currentState->next = nullptr;
```

```
        // Create and append the new state
        Node* newNode = new Node(newState);
        newNode->prev = currentState;
        currentState->next = newNode;

        // Move currentState to the new state
        currentState = newNode;
    }

    State* getCurrentState() {
        return currentState->state;
    }

    ~CanvasHistory() {
        Node* temp = head;

        while (temp != nullptr) {
            Node* nextNode = temp->next;
            delete temp->state;
            delete temp;
            temp = nextNode;
        }
    }
};

int main() {
    CanvasHistory history(new State(0));

    history.performAction(new State(1));
    history.performAction(new State(2));
    history.performAction(new State(3));
```

```
    cout << "Current: "
         << history.getCurrentState()->value << endl; // 3

    history.undo();
    cout << "After Undo: "
         << history.getCurrentState()->value << endl; // 2

    history.undo();
    cout << "After Undo: "
         << history.getCurrentState()->value << endl; // 1

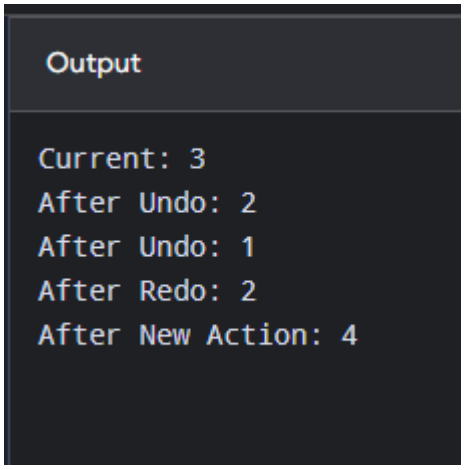
    history.redo();
    cout << "After Redo: "
         << history.getCurrentState()->value << endl; // 2

    // New action while at state 2.
    // State 3 is discarded.
    history.performAction(new State(4));

    cout << "After New Action: "
         << history.getCurrentState()->value << endl; // 4

    history.redo(); // Nothing to redo

    return 0;
}
```


Output
Current: 3
After Undo: 2
After Undo: 1
After Redo: 2
After New Action: 4

LAB 4

Circular Linked List

Activity 1:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

int main() {

    Node* first = nullptr;
    Node* last = nullptr;

    // Reserve space for a new node
    Node* p = new Node();

    // Place data into the node
    p->data = 15;

    // Make the necessary connections
    if (last != nullptr) {
        p->next = first;
        last->next = p;
        last = p;
    }
    else {
        first = p;
        last = p;
        p->next = first;
    }

    return 0;
}
```

Activity 2:

```
#include <iostream>
using namespace std;

struct Node {
    int key;
    int data;
    Node* next;
};

int main() {
    Node* pNode = new Node();
    Node* second = new Node();
    Node* third = new Node();

    pNode->key = 1;
    pNode->data = 15;

    second->key = 2;
    second->data = 25;

    third->key = 3;
    third->data = 30;

    pNode->next = second;
    second->next = third;
    third->next = pNode;   // circular

    // Accessing all nodes
    Node* p = pNode;

    if (p != nullptr) {
        do {
            cout << p->data << " ";
            p = p->next;
        } while (p != pNode);
    }

    cout << endl;

    // Searching for a key
    int givenKey = 2;
    p = pNode;

    if (p != nullptr) {
        do {
            if (p->key == givenKey) {
                cout << "Key found: " << p->data << endl;
```

```
                return 0;
            }

            p = p->next;

        } while (p != pNode);
    }

    cout << "Key not found";

    return 0;
}
```



Activity 3:

```
#include <iostream>
using namespace std;

struct Node {
    int key;
    int data;
    Node* next;
};

int main() {
    Node* pNode = new Node();
    Node* second = new Node();
    Node* third = new Node();

    pNode->key = 1;
    pNode->data = 15;

    second->key = 2;
    second->data = 25;

    third->key = 3;
    third->data = 30;

    pNode->next = second;
    second->next = third;
    third->next = pNode;
```

```
    // New node
    Node* p = new Node();
    p->key = 4;
    p->data = 20;

    int givenKey = 2;

    // Find node
    Node* q = pNode;
    Node* q1;

    do {
        q1 = q;
        q = q->next;

        if (q->key == givenKey)
            break;

    } while (q != pNode);

    // Insert before q
    if (q->key == givenKey) {
        q1->next = p;
        p->next = q;
    }

    return 0;
}
```

Activity 4:

```
#include <iostream>
using namespace std;

struct Node {
    int key;
    int data;
    Node* next;
};

int main() {
    Node* pNode = new Node();
    Node* second = new Node();
    Node* third = new Node();

    pNode->key = 1;
    pNode->data = 15;

    second->key = 2;
```

```
    second->data = 25;

    third->key = 3;
    third->data = 30;

    pNode->next = second;
    second->next = third;
    third->next = pNode;

    // Delete node with given key
    int givenKey = 2;

    Node* q = pNode;
    Node* q1;

    do {
        q1 = q;
        q = q->next;

        if (q->key == givenKey)
            break;

    } while (q != pNode);

    // Delete the node
    if (q->key == givenKey) {

        // Only one node in the list
        if (q == q->next) {
            pNode = nullptr;
        }
        else {
            q1->next = q->next;

            // If deleting the first node
            if (q == pNode)
                pNode = q1;
        }

        delete q;
    }

    return 0;
}
```

Activity 5:

```
#include <iostream>
using namespace std;
```

```
struct Node {
    int key;
    int data;
    Node* next;
};

int main() {
    Node* pNode = new Node();
    Node* second = new Node();
    Node* third = new Node();

    pNode->key = 1;
    pNode->data = 15;

    second->key = 2;
    second->data = 25;

    third->key = 3;
    third->data = 30;

    pNode->next = second;
    second->next = third;
    third->next = pNode;

    // Delete complete circular linked list
    Node* p = pNode;
    Node* p1;

    do {
        p1 = p;
        p = p->next;
        delete p1;
    } while (p != pNode);

    pNode = nullptr;

    return 0;
}
```

Lab Task 1:

```
#include <iostream>
using namespace std;

struct Node {
    int key;
    int data;
    Node* next;
};

int main() {
    Node* first = new Node();
    Node* second = new Node();
    Node* third = new Node();

    first->key = 1;
    first->data = 15;

    second->key = 2;
    second->data = 25;

    third->key = 3;
    third->data = 30;

    first->next = second;
    second->next = third;
    third->next = first;

    Node* p = first;

    if (p != nullptr) {
        do {
            cout << p->data << " ";
            p = p->next;
        } while (p != first);
    }

    return 0;
}
```



Lab Task 2:

```
#include <iostream>
using namespace std;

struct Node {
   int key;
   int data;
   Node* next;
};

int main() {
   Node* first = new Node();
   Node* second = new Node();
   Node* third = new Node();

   first->key = 1;
   first->data = 15;

   second->key = 2;
   second->data = 25;

   third->key = 3;
   third->data = 30;

   first->next = second;
   second->next = third;
   third->next = first;

   // New node
   Node* p = new Node();
   p->key = 4;
```

```
    p->data = 20;

    int givenKey = 2;

    // Find node
    Node* q = first;
    Node* q1;

    do {
        q1 = q;
        q = q->next;

        if (q->key == givenKey)
            break;

    } while (q != first);

    // Insert before q
    if (q->key == givenKey) {
        q1->next = p;
        p->next = q;
    }
    // Display
    Node* temp = first;
    do {
        cout << temp->data << " ";
        temp = temp->next;
    } while (temp != first);

    return 0;
}
```

LAB TASK 3:

```
#include <iostream>
using namespace std;

struct Node {
   int key;
   int data;
   Node* next;
};

int main() {
   Node* first = new Node();
   Node* second = new Node();
   Node* third = new Node();

   first->key = 1;
   first->data = 15;

   second->key = 2;
   second->data = 25;

   third->key = 3;
   third->data = 30;

   first->next = second;
   second->next = third;
   third->next = first;

   int givenKey = 2;

   Node* q = first;
```

```
Node* q1;

do {
   q1 = q;
   q = q->next;

   if (q->key == givenKey)
      break;

} while (q != first);

// Delete node
if (q->key == givenKey) {

   if (q == q->next) {
      first = nullptr;
   }
   else {
      q1->next = q->next;

      if (q == first)
         first = q1;
   }

   delete q;
}

// Display
if (first != nullptr) {
   Node* temp = first;
```

```
        do {
            cout << temp->data << " ";
            temp = temp->next;
        } while (temp  != first);
    }

    return 0;
}
```

